

Cross-Layer Transcoders on ESM-2 650M

A Research Reference: Method, Codebase, Bottlenecks & the Optimization Plan

Author: Sagnik Chatterjee · Collaborator: Saishradha Mohanty (Anna Green Lab, UMass Amherst)

Compiled 2026-07-14 · Fork of EleutherAI `sparsify` · Target model: ESM-2 650M → ESM-C 3B–6B

Contents

- | | |
|---|---|
| 1. Executive summary | 10. Distributed training & tensor parallelism |
| 2. Why interpretability for proteins | 11. Current state & the bottlenecks |
| 3. Method: SAE → Transcoder → CLT | 12. Optimization plan (the core work) |
| 4. The k-sparse mechanism in detail | 13. Evaluation framework |
| 5. ESM-2 650M as the substrate | 14. Biological validation track |
| 6. Codebase architecture | 15. Scaling to ESM-C |
| 7. Anatomy of one training step | 16. Open questions |
| 8. Cross-layer bookkeeping & detach/restore | 17. Reference library |
| 9. Loss functions | 18. Glossary & command cheat-sheet |

1 Executive summary

We train Cross-Layer Transcoders (CLTs) on the frozen activations of ESM-2 650M to decompose its computation into sparse, interpretable features and trace biological circuits across layers. The immediate objective is to make that training fast and memory-tractable so the recipe can scale to ESM-C 3B–6B.

A CLT is a sparse dictionary-learning model from Anthropic's *Crosscoders* line. Its encoder fires **once** at layer ℓ , and its decoder reconstructs the outputs of **every downstream target layer** $\ell+1 \dots \ell+N$. This makes the learned dictionary an explicit map of cross-layer computation — precisely what circuit tracing needs, and precisely what a per-layer SAE cannot represent.

The codebase is a fork of EleutherAI's `sparsify`, extended with CLT support, tensor parallelism (DTensor), bfloat16 training, ESM-2 hookpoints, and a combined KL+FVU loss. Basic SAE training already works at 650M; CLT training runs but is slow. Two bottlenecks dominate: (1) the frozen ESM-2 forward pass is recomputed every step, and (2) the CLT parameter count is enormous — ~89B parameters at the default config, dwarfing ESM-2 itself.

Two work tracks run in parallel. The *optimization track* (this document's focus, §11–12) profiles, adds activation caching, shrinks the CLT, and explores LoRA-style factorizations. The *applied track* (§14) takes trained CLTs and checks whether they recover the same circuits ProtoMech found on smaller models — the cheapest way to demonstrate scientific progress, and the recommended first task.

2 Why interpretability for proteins

Mechanistic interpretability (MI) asks what algorithm a network is actually running inside its weights. In language models this is a maturing program: we can isolate circuits (the IOI circuit in GPT-2) and decompose representations into monosemantic features with sparse autoencoders.

Protein language models are a high-value, under-explored MI target. ESM-2 (Lin et al., 2023, *Science*) predicts structure and function from sequence alone and underpins much of modern computational biology — yet *how* it computes contacts, folds, and functional sites is opaque. Extracting causal circuits converts predictions into testable hypotheses about protein biology, e.g. "this catalytic-motif feature at layer 6 gates this domain-recognition feature at layer 20."

Two recent results define the frontier we build on:

Work	Result	Scale
Nainani et al. 2025 NeurIPS MI Workshop, UMass	SAE latents in ESM-2 <i>causally mediate</i> contact prediction via a two-stage motif → domain circuit (motif detectors, layers 4–8; domain detectors, layers 12–24).	ESM-2, per-layer SAEs
ProtoMech Tsui, Talreja, Saeedi, Aghazadeh — ICML 2026	CLTs on ESM2-8M/35M recover 82–89% of original performance and discover circuits using <1% of the latent space.	ESM2-8M, ESM2-35M

The gap we fill: ProtoMech works at 8M–35M; Nainani uses per-layer SAEs that miss cross-layer computation. Nobody has trained CLTs at 650M, where representations are rich enough to matter biologically but training cost explodes. ESM-C's 3B–6B release makes the scaling question urgent — if we find the algorithmic bottlenecks at 650M, we can fix them before the larger models, where naive approaches are infeasible.

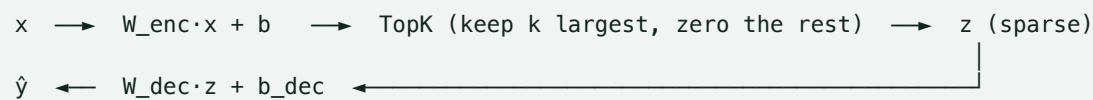
3 Method: SAE → Transcoder → CLT

All three learn an overcomplete, sparse dictionary. They differ in what they read and what they predict — a natural progression:

	reads	predicts (reconstructs)	
SAE	act at L	→ same act at L	(autoencode one site)
Transcoder	input of L	→ OUTPUT of L	(approximate a computation)
CLT (ours)	input of L	→ outputs of L ... L+N	(cross-layer computation)

Why CLT and not SAE? Real biological computation flows across layers. A per-layer SAE sees a frozen snapshot at each depth and cannot represent "feature A at layer 6 writes into feature B at layer 20." A CLT's encoder fires once at layer ℓ and its decoder emits reconstructions into every downstream target layer — so the learned dictionary is the cross-layer wiring. A transcoder (input→output of one layer) is more mechanistically interpretable than a residual-stream SAE because it approximates an actual computation; the CLT generalizes that across a window of layers.

4 The k-sparse mechanism in detail



- **TopK activation** (Gao et al., 2024) directly enforces sparsity — exactly k latents fire per token. No L1 penalty, no sparsity coefficient to tune. This repo defaults to TopK (`activation="topk"`) with `batchtopk` and `groupmax` as alternatives.
- **Expansion factor** sets dictionary size: `num_latents = expansion_factor × d_model`. For ESM-2 650M (`d_model=1280`), `expansion_factor=128` → **163,840 latents**.
- **Sparsity level** k is the number of active latents per token (typically `num_latents/1000` to `/500`). Higher k improves reconstruction but reduces interpretability.
- The repo uses `rtopk` (randomized TopK) and custom Triton kernels for the sparse encoder backward — see `fused_encoder.py`.

5 ESM-2 650M as the substrate

Property	Value	Consequence for the CLT
Transformer layers	33	Up to 33 CLTs; cross-layer windows span meaningful biology
<code>d_model</code>	1280	Encoder/decoder width — the source of the parameter blow-up
Attention heads	20	—
Max context	1022 residues	Long sequences → many tokens/batch
Special token	CLS (no BOS)	Masked in training via <code>--filter_bos=True</code>

A CLT sits at each layer's MLP, reading its input and reconstructing outputs across a window of `cross_layer=N` downstream layers. The ESM-2 weights are frozen throughout; only the CLTs are trained.

6 Codebase architecture

The fork is small and legible. Four files carry the architecture — read them in this order:

File	Role
<code>sparsify/config.py</code>	<code>SparseCoderConfig</code> (aka <code>SaeConfig</code>) + <code>TrainConfig</code> . Every knob. Start here.
<code>sparsify/sparse_coder.py</code>	<code>SparseCoder</code> (the CLT), <code>MidDecoder</code> , <code>ForwardOutput</code> . Encode / TopK / multi-target decode.
<code>sparsify/runner.py</code>	<code>CrossLayerRunner</code> — orchestrates cross-layer encode/decode and the gradient detach/restore mechanism.
<code>sparsify/trainer.py</code>	<code>Trainer</code> — hooks ESM-2, runs the loop, computes losses, checkpoints, handles DTensor TP.

Supporting files: `fused_encoder.py` (fused Triton kernels for the sparse encoder backward and COO decoder), `kernels.py`, `data.py` (`chunk_and_tokenize`, `MemmapDataset` — the infrastructure for activation caching), `__main__.py` (CLI entry point building `RunConfig`), and optimizers `muon.py` / `sign_sgd.py`.

What the fork adds over base `sparsify`: CLT support; tensor parallelism via DTensor; bfloat16 training; input hookpoints inserted before layernorm; combined KL+FVU loss (arXiv:2503.17272); ESM-2 support (CLS masking, `input_ids` handling); sweep infrastructure in `optimization/analyze_sweeps.py`.

6.1 Key configuration knobs

Flag	Default	Meaning
<code>--activation</code>	topk	topk / batchtopk / groupmax
<code>--expansion_factor</code>	32	dictionary size = EF × d_model
<code>--k</code>	—	latents kept per token
<code>--transcode</code>	False	transcoder mode (input→output) vs SAE
<code>--skip_connection</code>	False	linear skip around the transcoder
<code>--cross_layer</code>	0	number of downstream targets (N)
<code>--coalesce_topk</code>	none	none / concat / per-layer / group
<code>--topk_coalesced</code>	False	apply TopK to the coalesced values
<code>--loss_fn</code>	fvu	ce / fvu / kl / kl-fvu
<code>--kl_coeff</code>	1.0	0.0 = auto-scale KL to FVU magnitude
<code>--optimizer</code>	signum	adam / adam8 / muon / signum
<code>--layer_stride</code>	1	train every Nth layer (exclusive with <code>--layers</code>)
<code>--tp</code>	—	tensor-parallel degree

7 Anatomy of one training step

```
batch of protein sequences
  |
  ▼
ESM-2 650M forward pass (FROZEN, recomputed every step) ← bottleneck #1
forward hooks fire at each hooked MLP module
  | (x = MLP input, y = MLP output captured by the hook)
  ▼
CrossLayerRunner.encode(x): z = TopK(W_enc·x + b) → MidDecoder holds (x, z, idx)
  |
  ▼
CrossLayerRunner.decode(): for each open source layer:
                           ŷ_target += W_dec[target]·z_source
                           coalesce_topk ∈ {none, concat, per-layer, group}
  |
  ▼
Loss: FVU (reconstruction) | KL (faithfulness) | KL-FVU (combined)
detach/restore backward across layers → optimizer step
```

8 Cross-layer bookkeeping & detach/restore

8.1 The runner's open-source dictionary

One source layer writes into *many* targets. `CrossLayerRunner` keeps a dict of "open" `MidDecoder` objects. When layer ℓ 's output arrives, every still-open source that reaches ℓ contributes its decode; a source is deleted once it emits its last target (`will_be_last`). Paraphrased core loop:

```
for i, (hookpoint, layer_mid) in enumerate(self.outputs.items()):
    if detach_grad:
        layer_mid.detach()                # cut graph → per-layer backward
        candidate_indices.append(layer_mid.latent_indices + i * num_latents)
        candidate_values.append(layer_mid.current_latent_acts)
        out = layer_mid(y, ...)            # decode this source into target y
        if hookpoint != module_name:
            output += out.sae_out          # accumulate cross-layer contributions
```

`coalesce_topk` controls how latents from multiple sources combine at a target: `none` (each source decodes independently and sums — standard CLT); `concat` (tie all sources into one shared dictionary, TopK over the union); `per-layer` / `group` (per-target tied variants).

8.2 The detach/restore backward trick

Backprop through a 33-layer CLT would require holding the entire graph at once. Instead:

1. During forward, latent activations are **detached** (`z.detach()`; `z.requires_grad=True`) so each layer's subgraph is independent.
2. `layer_mid(y).backward()` runs per-layer.
3. `runner.restore()` re-attaches the originals and calls `original_activations.backward(grad)` to push gradients back into the encoder.

Cost: this bounds memory but is a *custom backward*, which prevents `torch.compile` from tracing the full loop and serializes the per-layer backward passes so they cannot be fused. Revisiting whether this is still necessary at 650M scale is a real optimization lever (§12, Step 4).

9 Loss functions

Loss	What it measures	ESM-2 passes / step
fvu	Fraction of Variance Unexplained — pure reconstruction quality	1
kl	KL between ESM-2's original logits and logits when the CLT replaces the MLP — <i>faithfulness</i> to the model's computation	2 (clean + dirty)
kl-fvu	Combined (arXiv:2503.17272): reconstruct well <i>and</i> stay faithful. <code>kl_coeff=0.0</code> auto-scales KL to FVU magnitude	2
ce	End-to-end cross-entropy on final logits	1–2

Which to use? For circuit discovery in pLMs, faithfulness to ESM-2's internal computation matters more than token-prediction CE, so `kl-fvu` is the working default — at the cost of the two-pass overhead, which makes activation caching even more valuable.

CLS handling: ESM-2 has no BOS token, so the trainer masks the CLS token (`--filter_bos=True`) to avoid contaminating position-specific losses — important for contact prediction, which uses position-specific outputs.

10 Distributed training & tensor parallelism

`--tp=N` shards each CLT across N GPUs using PyTorch **DTensor** (Shard/Replicate placements with explicit `.redistribute()` collectives). This is necessary because a single CLT layer's weights can exceed one GPU's VRAM. Alternatives:

- **DDP** (default) — replicate each CLT on every GPU. Simple, memory-inefficient.
- `--distribute_modules` — allocate different layers' CLTs to different GPUs (layer-parallel). Memory-efficient when a single CLT fits on one GPU; requires the GPU count to divide the layer count.
- `--tp=N` — shard within each CLT. Needed when one CLT exceeds a GPU. Each `.redistribute()` triggers an all-gather or reduce-scatter — high latency on PCIe (bottleneck #3).

11 Current state & the bottlenecks

Basic SAE training works on ESM-2 650M. CLT training runs but is slow and memory-heavy. In impact order:

Bottleneck 1 — No activation caching (biggest)

`Trainer.fit()` runs the full frozen ESM-2 650M forward pass every step, even though its weights never change. For `kl-fvu` it is **two** passes per step. Every hyperparameter sweep re-runs ESM-2 over the same sequences from scratch. The base `sparsify` README itself lists "caching activations" as an unimplemented TODO.

Rough cost: ESM-2 650M at bf16 processes $\sim 1\text{--}2$ tokens/ms/GPU. A batch of $32 \text{ seqs} \times 1022 \text{ tokens} \approx 32.7\text{K}$ tokens $\rightarrow \sim 16\text{--}32$ ms/GPU *before any CLT compute*. Over 100K steps that is $\sim 1,600\text{--}3,200$ GPU-seconds spent purely recomputing a frozen model — repeated for every sweep run.

Bottleneck 2 — Enormous CLT parameter count

At `expansion_factor=128`, `d_model=1280`, `cross_layer=12`:

encoder / layer	: $163,840 \times 1,280$	$\approx 210\text{M}$ params	
decoders / layer	: $210\text{M} \times 12$ targets	$\approx 2.5\text{B}$ params	
per CLT layer	:	$\approx 2.7\text{B}$ params	
all 33 layers	:	$\approx 89\text{B}$ params	← dwarfs ESM-2 (650M) itself

That is why real runs quietly reduce scale. The same recipe on ProtoMech's ESM2-35M (`d_model=480`) is $\sim 121\times$ smaller — the $\sim 11\times$ width increase to 650M drives a $\sim 121\times$ parameter increase.

Bottleneck 3 — TopK at scale, DTensor comms, custom backward

- TopK over $163,840$ latents $\times \sim 32\text{K}$ tokens/batch = billions of comparisons per step.
- Every hooked layer triggers DTensor all-gather / reduce-scatter, catastrophic on PCIe.
- The detach/restore custom backward blocks `torch.compile` fusion.
- Secondary: shuffling very large HF Arrow datasets can bottleneck data loading.

12 Optimization plan (the core work)

Ordered by impact-to-effort. This is the operational summary; the exhaustive version with storage math is in `docs/objective1_writeup.md`.

Step 0 — Profile before changing anything (1–3 days)

Get ground-truth numbers first. Wrap ~ 10 loop steps in `torch.profiler.profile(activities=[CPU, CUDA], with_stack=True)`, export a Chrome trace, and cross-check with `nsys profile`. Measure: time in ESM-2 forward vs CLT forward vs CLT backward; memory split (frozen weights / CLT weights /

optimizer states); GPU utilization; peak vs allocated CUDA memory; steps/sec as a function of layers trained (1 vs 5 vs 10 vs 33).

Decision gate: the profile tells you whether you're forward-bound (small CLT) or backward-bound (large expansion factor). Do not build anything until you know which regime you're in.

Step 1 — Activation caching (highest leverage; 1–2 weeks)

Precompute and memory-map ESM-2 activations so the frozen forward is not recomputed each step.

`MemmapDataset` already exists in `sparsify/data.py`. Per sequence, dump: MLP input (transcoder input), MLP output (transcoder target), and residual-stream activations at layer boundaries. Expected **3–10× speedup**; sweeps become nearly free because activations are computed once.

The catch is storage. A full cache (33 layers × both directions × 1M seqs) reaches tens of TB even in bf16. Two practical routes:

- **Cache a layer subset** (e.g. every 4th layer = 9 layers) and/or a smaller dataset (100K seqs).
- **Streaming ring buffer** — pre-compute N batches ahead in a background thread so ESM-2 leaves the gradient path without materializing the full cache to disk. Prefer this at ESM-C scale.

Step 2 — Shrink the CLT (3–5 days)

Reproduce ProtoMech-style quality at reduced scale *before* scaling up:

- `--expansion_factor 32` (num_latents 40,960; ~16× fewer params than EF=128).
- `--cross_layer 4 – 6` initially; note Nainani's circuit spans layers ~4→24, so full circuit capture may need ≥16 — test empirically.
- `--layer_stride 2` (train every 2nd layer; already supported).
- Sweep `k` for the new size (≈40–80 here).

Step 3 — LoRA-style factorization (the direction Saishradha flagged; 2–3 weeks)

- **Decoder:** $W_{\text{dec_target}} = W_{\text{dec_shared}} + \alpha \cdot (A_{\text{target}} \cdot B_{\text{target}})$ — a shared base dictionary plus small per-target low-rank adapters. Memory savings scale with `cross_layer`. Cleanest to try first.
- **Encoder:** low-rank or **product-key memory (PKM)** factorization (Lample et al., 2019). Split the latent space into two $\sqrt{\text{num_latents}}$ groups with sub-encoders; the full latent (i,j) is activated by an outer-product structure. Potentially ~100–400× fewer encoder params at some expressivity cost.

Benchmark each against full-rank on `val_mean_fvu` and `val_replacement_ce_increase`. Open question: does factorization wash out target-specific features that matter for circuit fidelity?

Step 4 — Training-loop wins (1–2 weeks)

- **`torch.compile` coverage.** Encoder/decoder forwards are already compiled; the custom `FusedEncoder` backward blocks full-loop compilation. Audit whether it is actually faster than plain autograd at 650M (PyTorch 2.x sparse-grad has improved) — if not, dropping it unlocks compilation.

- **8-bit Adam** (`--optimizer adam8`) cuts optimizer state from ~ 16 to ~ 6 bytes/param; keep master math in fp32.
- **Activation checkpointing on the CLT forward** — cheap relative to ESM-2, so recomputing in backward is a good memory trade.
- **TP tuning** — overlap NCCL AllReduce with the next layer's forward; consider `--tp 1` + `--distribute_modules` when a CLT fits on one GPU; or reduce `tp` and raise `--grad_acc_steps`.

Step 5 — Sweep with the existing infra (1–2 weeks)

`optimization/analyze_sweeps.py` ranks W&B runs by `val_mean_fvu`, then `val_replacement_ce_increase`, then `val_mean_l0`. Run on a layer subset (`--layers 8 16 24`) before full 33-layer training. Prefer Optuna TPE over grid search.

Parameter	Values to sweep
<code>expansion_factor</code>	16, 32, 64
<code>k</code>	16, 32, 64
<code>lr</code>	1e-4, 2e-4, 5e-4
<code>optimizer</code>	adam, signum
<code>cross_layer</code>	4, 8, 12
<code>coalesce_topk</code>	none, concat

On learning Triton / kernels: per the June 30 sync, low-level kernel optimization is valuable long-term but *not* a prerequisite to make progress now. Saishradha's explicit advice was to avoid "panic mode" reading kernel docs — the config-level, caching, and factorization wins deliver most of the speedup without hand-writing kernels. Treat kernel authoring as an optional later deep-dive, and lead with the applied CLT circuit work (§14).

13 Evaluation framework

13.1 Training-quality metrics

Metric	Symbol	Target	Meaning
Validation FVU	<code>val_mean_fvu</code>	< 0.05	Fraction of variance unexplained across layers
Replacement CE increase	<code>val_replacement_ce_increase</code>	< 0.1 nats	CE increase when CLT replaces ESM-2 – the faithfulness gold standard
Mean L0	<code>val_mean_l0</code>	$\approx k$	Average active features per token
Dead latent %	<code>dead_pct</code>	< 5%	% of features that never activate
Density mean	<code>mean_density_mean_ema</code>	0.001–0.01	Feature usage density

13.2 Efficiency metrics

Metric	Baseline	Target after optimization
Training throughput (tokens/sec)	TBD (profile first)	3–10× improvement
Time to 5% FVU	TBD	< 12 GPU-hours
Peak GPU memory	TBD	< 40 GB / GPU
Steps per second	TBD	2–5× improvement

13.3 Biological-validation metrics

Metric	Meaning
Protein family F1 (ProtoMech protocol)	Classification accuracy with CLT as replacement model
Contact prediction AUROC (Nainani protocol)	Accuracy under causal circuit patching
ProteinGym Spearman R	Mutation-effect prediction
Circuit compression ratio	% of latent space needed to retain 80% of task performance

14 Biological validation track (recommended first task)

This is the cheapest way to show scientific results and was Saishradha's recommended first step. Once any 650M CLT is trained:

1. **Reproduce ProtoMech's mutation-effect-prediction setup.** ProtoMech does supervised mutation-effect prediction for **two proteins** (see [papers/2602.12026v2.pdf](#)). Run our CLTs on those two and

check whether they recover **the same circuits** — the first sanity check. Hypothesis: they should, and as a 650M model (vs their 35M), ours may surface richer / more features.

2. **Then move to the unsupervised task.** Saishradha is writing a paper on an *unsupervised* mutation-effect-prediction task; the medium-term goal is to translate our CLTs to that setting (needs a slight change from the supervised setup).
3. **Reproduce & sharpen Nainani's motif→domain circuit** via activation patching — CLTs model the actual cross-layer wiring, so the circuit should appear at higher fidelity. Candidate proteins: MetXA (P45131), TOP2 (P06786).
4. **ProteinGym** — test whether circuit-relevant CLT features alone predict mutation effects, validating the circuit's causal role.

Cadence: weekly update to Saishradha on Tuesdays.

15 Scaling to ESM-C 3B–6B

Once the 650M pipeline is validated, apply the same optimizations to ESM-C. Key differences:

- ESM-C 3B: `d_model` $\approx$ 2048–2560, $\sim$ 36 layers. ESM-C 6B: `d_model` $\approx$ 3072–4096, $\sim$ 48 layers.
- Requires 8+ A100 80GB GPUs minimum for model + CLT weights.
- Activation caching is even more critical (model forward $\sim$ 4–9 $\times$ slower than 650M) — streaming likely mandatory.
- May require `--expansion_factor 16` or `8` initially; `--tp 8` to fit CLT weights across GPUs.

16 Open questions

- **Is KL-FVU the right objective for biological circuits**, versus pure faithfulness (KL)? Faithfulness to ESM-2's computation may matter more than reconstruction for circuit claims.
- **How much cross-layer reach is enough?** Nainani's circuit spans layers $\sim$ 4→24; does `cross_layer` need to be ≥ 16 to capture it, and what does that cost?
- **Do factorized decoders preserve circuit fidelity**, or wash out target-specific features?
- **Caching vs streaming** at ESM-C scale, where full caches are tens of TB.
- **Per-MLP or per-residual-stream** hookpoints? Transcoders on MLP in/out are more mechanistically interpretable but require careful hookpoint choice.
- **The right biological gold standard** for "this circuit is real" — contact AUROC under patching? ProteinGym Spearman?

17 Reference library (in papers/)

Reference	File	Relevance
Crosscoders, Anthropic 2024	transformer-circuits.pub	The CLT method origin
ProtoMech, Tsui et al., ICML 2026	2602.12026v2.pdf	CLTs on ESM2-8M/35M — direct predecessor
Nainani et al. 2025, NeurIPS MI Wksp	117_Mechanistic_evidence_that_.pdf	SAE motif→domain circuit for contacts
Sparse Feature Circuits, Marks et al., ICLR 2025	2403.19647v3.pdf	Causal circuit discovery methods
IOI circuit, Wang et al. 2022	2211.00593v1.pdf	Foundational circuit analysis
ESM / zero-shot mutation effects, Meier/Rives	NeurIPS-2021-language-models-...pdf	ESM-2 background
ProteinGym, NeurIPS 2023	NeurIPS-2023-proteingym-...pdf	Fitness-prediction benchmark
Scaling & evaluating SAEs, Gao et al. 2024	arXiv:2406.04093	The TopK recipe behind base sparsify
KL+FVU training	arXiv:2503.17272	The combined-loss method

Related repo docs: docs/ONBOARDING.md , docs/objective1_writeup.md (full plan), docs/presentation_xiaocong.md (technical walkthrough), root CLAUDE.md / README.md (build/run/train commands).

18 Glossary & command cheat-sheet

18.1 Glossary

Term	Meaning
CLT	Cross-Layer Transcoder — encoder fires at ℓ , decodes into $\ell+1 \dots \ell+N$
SAE	Sparse Autoencoder — reconstructs the same activation it reads
Transcoder	Reads a layer's input, predicts its output (approximates a computation)
FVU	Fraction of Variance Unexplained — reconstruction error metric
TopK	Activation keeping the k largest pre-activations, zeroing the rest
Hookpoint	Module name (glob-matched) where the CLT is inserted
Coalesce	How latents from multiple source layers combine at a target
Replacement CE	CE increase when CLT reconstructions replace ESM-2's MLP
Dead latent	A feature that never activates across the dataset

18.2 Commands

```
# Environment
uv venv --seed && uv sync
pip install -e .[dev]
pre-commit install
pytest tests/

# Reduced-scale CLT smoke run (see §12 Step 2)
uv run python -m sparsify facebook/esm2_t33_650M_UR50D <DATASET> --run_name smoke \
  --transcode=True --skip_connection=True --filter_bos=True \
  --expansion_factor 32 --k 32 --cross_layer 4 --layer_stride 2 \
  --loss_fn kl-fvu --optimizer adam --lr 1e-4

# Distributed (layer-parallel, memory-efficient)
torchrun --nproc_per_node gpu -m sparsify facebook/esm2_t33_650M_UR50D \
  --distribute_modules --batch_size 1 --layer_stride 2 --grad_acc_steps 8

# Analyze a sweep
python optimization/analyze_sweeps.py
```

Set `WANDB_ENTITY` for logging. Checkpoints land in `checkpoints/<run_name>/`. Confirm the exact ESM-2 HF model id and dataset path with Saishradha before the first full run.